

01 Generative AI – Fundamentals

1.1 What is Generative AI?

Generative AI (Gen AI) is a category of artificial intelligence that can create new content — text, images, audio, video, code, and more — based on patterns learned from large amounts of training data.

Unlike traditional AI that classifies or predicts, Generative AI generates novel outputs that did not exist before.

Key Characteristics

- Learns from massive datasets (books, websites, code, images)
- Generates new, original content rather than just classifying input
- Uses probability to predict the most likely next token / pixel / frame
- Can be fine-tuned for specific tasks and domains

Type	Traditional AI
Goal	Classify / predict existing patterns
Output	Label, score, or decision
Example	Spam detection, fraud scoring
Training	Supervised labels required

1.2 How LLMs Work

Large Language Models (LLMs) are the core technology behind most Gen AI text tools. Understanding how they work helps you use them effectively.

Tokens

A token is the smallest unit of text that an LLM processes. Tokens are not exactly words — they can be parts of words, punctuation, or spaces.

Concept	Explanation
Token	A chunk of text — roughly 3-4 characters or ~0.75 words on average
Tokenization	The process of splitting input text into tokens before feeding to the model
Example	'Hello World' = 2 tokens. 'Unbelievable' might be 3-4 tokens
Why it matters	API costs and context limits are measured in tokens, not words
Approx ratio	1,000 tokens ≈ 750 words ≈ ~1.5 pages of text

Context Window

The context window is the maximum number of tokens an LLM can process at one time — including both the input (prompt) and the output (response).

Model	Context Window (approx.)
GPT-3.5	4,096 tokens (~3,000 words)
GPT-4	8K to 128K tokens (varies by version)
Claude 3 Sonnet	200,000 tokens (~150,000 words)
Gemini 1.5 Pro	1,000,000 tokens (1M token window)

 If your conversation exceeds the context window, the model forgets earlier parts of the conversation. This is why very long chats sometimes lose context.

Transformers – The Architecture Behind LLMs

Transformers are the neural network architecture that powers virtually all modern LLMs. Introduced in the 2017 paper 'Attention Is All You Need'.

Component	What it Does	Simple Analogy
Embedding	Converts tokens into numerical vectors (numbers)	Turning words into coordinates on a map
Self-Attention	Figures out which words are related to each other in context	Highlighting related words while reading a sentence
Feed-Forward Layers	Applies learned patterns and transformations	Processing the highlighted information
Softmax Output	Converts final scores into probabilities for next token	Picking the most likely next word from a ranked list

 Self-Attention is what makes Transformers so powerful — unlike older models (RNNs), they can relate any word to any other word in the sentence regardless of distance.

How Text Generation Works (Step by Step)

1. Input text (prompt) is split into tokens
2. Tokens are converted to numerical embeddings
3. Embeddings pass through many Transformer layers (self-attention + feed-forward)
4. Model outputs a probability distribution over all possible next tokens
5. One token is sampled based on those probabilities
6. That token is appended and the process repeats until generation ends

 Temperature controls randomness. Low temperature (0.1) = more focused and predictable. High temperature (1.0) = more creative but less consistent.

1.3 Popular Gen AI Models

Model	Company	Key Strengths
GPT-4 / GPT-4o	OpenAI	Multimodal (text+image), strong reasoning, widely used API
Claude 3.5 Sonnet	Anthropic	200K context, safety-focused, excellent for long documents
Gemini 1.5 Pro	Google DeepMind	1M token context, native multimodal, integrated with Google Workspace
Llama 3	Meta	Open-source, free to run locally, great for self-hosted apps
Mistral	Mistral AI	Open-source, efficient, strong coding and reasoning
Copilot	Microsoft / OpenAI	Integrated into VS Code, GitHub, Office 365
DALL-E 3	OpenAI	Text-to-image generation
Stable Diffusion	Stability AI	Open-source image generation, can run locally

1.4 Prompt Engineering Basics

Prompt Engineering is the practice of designing and optimizing text inputs (prompts) to get the best possible outputs from an LLM. It is a critical skill for working with Gen AI systems.

Core Prompting Techniques

Technique	Description
Zero-Shot	Ask directly without any examples
One-Shot	Provide one example before asking
Few-Shot	Provide multiple examples
Chain of Thought	Ask model to reason step by step
Role Prompting	Give the model a persona / role
System Prompt	Set behaviour and context upfront

Elements of a Good Prompt

- Context — Who are you? What is the situation?
- Task — Clearly state what you want the model to do
- Format — Specify how you want the output (bullet points, JSON, table, paragraph)
- Constraints — Word limit, tone, language, style
- Examples — Show the model what a good answer looks like

```
# Bad Prompt:
Tell me about testing

# Good Prompt:
You are a senior software tester.
Explain API testing to a beginner in simple language.
Cover: what it is, why it matters, and 3 key techniques.
Format the response with headings and bullet points.
Keep it under 300 words.
```

1.5 Gen AI Use Cases

Domain	Use Case	Example Tool
Software Testing	Generate test cases, test data, bug reports	ChatGPT, Copilot
Development	Code generation, code review, debugging	GitHub Copilot, Claude
Documentation	Write API docs, user manuals, changelogs	Notion AI, Claude
Customer Support	Chatbots, FAQ answering, ticket routing	GPT-4, Dialogflow
Data Analysis	Summarize reports, explain data trends	Gemini, Claude
Content Creation	Write blogs, emails, marketing copy	Jasper, ChatGPT
Education	Explain concepts, create quizzes, tutoring	Khan Academy AI
HR & Recruitment	Screen resumes, write JDs, interview prep	HireVue AI
Healthcare	Medical record summarization, diagnosis assist	Med-PaLM 2
Legal	Contract review, clause extraction	Harvey AI

1.6 Fundamental Concepts of Gen AI and LLMs

Concept	Explanation
Pre-training	Model trained on massive datasets (billions of web pages, books, code) to learn language patterns
Fine-tuning	Additional training on a smaller domain-specific dataset to specialize the model's behaviour
RLHF	Reinforcement Learning from Human Feedback — humans rate outputs to align model with desired behavior
Inference	When the trained model generates a response (applying what it learned to new inputs)
Hallucination	Model confidently produces incorrect or made-up information — a key limitation
Embeddings	Numerical vector representations of text used to measure semantic similarity
RAG	Retrieval Augmented Generation — model retrieves relevant documents before generating response
Parameters	Weights inside the model learned during training. GPT-4 has ~1 trillion parameters (estimated)
Multimodal	Model that can process multiple types of input — text, image, audio, video

1.7 Generative AI: Impact, Considerations & Ethical Issues

Positive Impact

- ✓ Massive productivity gains — automates repetitive writing, coding, analysis tasks
- ✓ Democratizes access to expertise — anyone can get expert-level help
- ✓ Accelerates research and innovation across all fields
- ✓ Enables personalized learning and healthcare at scale

Key Concerns & Ethical Issues

Issue	Description
Bias	Models inherit biases from training data — may produce biased outputs
Hallucinations	Model generates false facts with high confidence
Misinformation	Easy to generate fake news, deepfakes, and propaganda at scale
Copyright	Training on copyrighted data and reproducing it in outputs
Privacy	Training data may include personal information

Job Displacement	Automation of tasks previously done by humans
Environmental Impact	Training LLMs consumes enormous compute and energy
Security	Prompt injection, jailbreaking, and model manipulation

1.8 Foundation of Gen AI in Software Testing

Gen AI is transforming how software testing is done — from test design to execution and closure. It acts as an AI co-pilot for QA engineers.

Where Gen AI Helps in the Test Lifecycle

Phase	Traditional Approach	With Gen AI
Test Planning	Manual analysis of requirements	AI reads requirements and suggests test strategy
Test Case Design	Manually write test cases	Generate test cases from user stories or specs
Test Data	Manually create or use static data	Generate realistic synthetic test data on demand
Execution	Run tests, manually check results	AI interprets failures and suggests root causes
Bug Reports	Manually write bug descriptions	AI generates detailed bug reports from logs
Closure	Manual test summary reports	AI generates release summaries and coverage reports

1.9 Test Design and Data Preparation with Gen AI

Generating Test Cases with Gen AI

You can paste a user story or requirement into an LLM and ask it to generate test cases:

```
Prompt:
You are a QA engineer. Given the following user story, generate detailed
test cases covering positive, negative, and edge cases.
Format: Test ID | Title | Steps | Expected Result

User Story:
As a user, I want to log in with my email and password so I can access my account.
```

Generating Test Data with Gen AI

- Generate realistic fake names, emails, phone numbers, addresses

- Generate edge-case data: very long strings, special characters, null values
- Generate data in different formats: CSV, JSON, SQL INSERT statements
- Generate boundary values: min, max, just-below-min, just-above-max

```
Prompt:
Generate 10 realistic test user records in JSON format with fields:
id, name, email, phone, age, country.
Include 2 records with invalid email formats and 1 with age = 0.
```

1.10 Execution and Closure using Gen AI

During Test Execution

- Paste error logs into LLM and ask: 'What does this error mean and how do I fix it?'
- Ask AI to explain a failing test assertion in plain English
- Ask AI to suggest which tests to prioritize based on risk area
- Use AI to review existing automation scripts for improvement

Test Closure with Gen AI

- Generate test summary reports from execution results
- Draft release notes based on what was tested and what passed
- Create metrics summaries: pass/fail rate, coverage, defect density
- Generate retrospective notes and lessons learned

```
Prompt:
Here are my API test execution results:
- Total tests: 120
- Passed: 108, Failed: 12
- Failed tests: [list test names]
- Environment: Staging

Generate a professional test closure report suitable for sharing with
stakeholders.
```

02 API Integration with Gen AI

2.1 How Gen AI APIs Work

Gen AI models are accessed through REST APIs. You send a request with your prompt, the model processes it, and returns the generated response.

Provider	API
OpenAI (ChatGPT)	OpenAI API
Anthropic (Claude)	Messages API
Google (Gemini)	Gemini API
Meta (Llama)	Various (via Groq, Together AI)

2.2 Calling a Gen AI API – Example

OpenAI Chat Completion API

```
import openai

client = openai.OpenAI(api_key='YOUR_API_KEY')

response = client.chat.completions.create(
    model='gpt-4o',
    messages=[
        {'role': 'system', 'content': 'You are a helpful QA assistant.'},
        {'role': 'user', 'content': 'Write 5 test cases for a login API.'}
    ],
    temperature=0.7,
    max_tokens=500
)

print(response.choices[0].message.content)
```

Anthropic Claude API

```
import anthropic

client = anthropic.Anthropic(api_key='YOUR_API_KEY')

message = client.messages.create(
    model='claude-sonnet-4-5',
    max_tokens=1024,
    messages=[
```

```

        {'role': 'user', 'content': 'Explain API testing in 3 bullet points.'}
    ]
)

print(message.content[0].text)

```

Key API Parameters

Parameter	Description
model	Which LLM to use (e.g., gpt-4o, claude-sonnet-4-5)
messages	List of conversation turns with role: system / user / assistant
temperature	Controls randomness. 0 = deterministic, 1 = creative. Default ~0.7
max_tokens	Maximum number of tokens in the response
top_p	Nucleus sampling — controls diversity of output (alternative to temperature)
stream	If true, response is streamed token by token (like typing effect)
system	OpenAI system prompt — sets the assistant's persona and instructions

2.3 Hands-on: Simple Chatbot using Gen AI API

A simple command-line chatbot that maintains conversation history across turns:

```

import openai

client = openai.OpenAI(api_key='YOUR_API_KEY')

def chat(messages):
    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=messages,
        temperature=0.7
    )
    return response.choices[0].message.content

# Conversation history — includes system prompt
history = [
    {'role': 'system', 'content': 'You are a helpful AI assistant for QA engineers.'}
]

print('Chatbot is ready! Type quit to exit.')
while True:
    user_input = input('You: ')
    if user_input.lower() == 'quit':
        break
    history.append({'role': 'user', 'content': user_input})
    reply = chat(history)
    history.append({'role': 'assistant', 'content': reply})

```

```
print(f'Bot: {reply}')
```

 The history list is what gives the chatbot memory within a session. Each turn (user + assistant) is appended, so the model always has full conversation context.

2.4 Limitations & Hallucinations

Limitation	Description
Hallucination	Model generates false info confidently
Knowledge Cutoff	Model doesn't know events after training date
Context Limit	Can't process more than context window
Inconsistency	Same prompt may produce different outputs
No Memory	Stateless — forgets between API calls
Prompt Injection	Malicious input can hijack model behavior
Slow for Complex Tasks	Multi-step reasoning takes time
Cost	API calls are billed per token

2.5 Ethics & Responsible AI

Core Principles of Responsible AI

Principle	What it Means
Fairness	AI should not discriminate based on race, gender, religion, or other protected attributes
Transparency	Users should know when they are interacting with AI, not a human
Accountability	Developers and organizations are responsible for AI outputs and their impact
Privacy	Personal data used in AI systems must be protected and handled lawfully
Safety	AI should not cause harm — physical, psychological, societal
Inclusivity	AI should work well for all people, not just majority demographics
Human Oversight	Humans must remain in the loop for high-stakes decisions

Practical Responsible AI in Testing

- Never use real customer PII (personal data) as test data — always anonymize

- Test AI models for bias across different demographic groups
- Document and disclose when AI is used to generate test cases or code
- Review AI-generated test outputs before using them — do not blindly trust
- Follow your organization's AI usage policy

03 Agentic AI

3.1 What is Agentic AI?

Agentic AI refers to AI systems that can take autonomous actions to accomplish goals — they don't just respond to a single prompt, they plan, reason, use tools, and execute multi-step tasks with minimal human intervention.

Feature	Traditional LLM (Chatbot)
Interaction	Single-turn prompt → response
Memory	No memory (stateless)
Tools	Cannot use external tools
Decision-making	One-step output
Example	'Write a test case'

3.2 Agents vs Chatbots

Aspect	Chatbot	AI Agent
Goal	Answer questions	Accomplish tasks
Autonomy	Low — waits for each prompt	High — takes initiative
Memory	None (resets each chat)	Short + long term memory
Tool Use	None (text only)	Web, code, files, APIs
Planning	None	Multi-step planning
Loop	Request → Response	Plan → Act → Observe → Reflect → Repeat
Example	Customer support bot	Research agent, coding agent, testing agent

3.3 Agent Architecture

The Core Loop: Plan → Act → Observe → Reflect

Step	Description
Planning	Agent breaks down the goal into sub-tasks

Reasoning	Agent decides what action to take next based on current state
Tool Use	Agent calls external tools to gather information or take actions
Observation	Agent receives result from tool and updates its understanding
Reflection	Agent evaluates progress, adjusts plan if needed

Agent Components

- Brain (LLM) — The reasoning engine that plans, decides, and generates actions
- Memory — Stores context, history, and learned information
- Tools — External capabilities the agent can invoke (APIs, search, code interpreter)
- Perception — Input from the environment (API responses, file contents, user messages)
- Action — Outputs that affect the world (calling an API, writing a file, sending an email)

3.4 Function Calling & Tool Use

Function Calling (also called Tool Use) lets an LLM invoke predefined functions/tools as part of its response. The model decides when to call a tool and with what arguments.

How Function Calling Works

7. You define available tools/functions in the API call
8. The model reads the user request and decides if a tool is needed
9. Model returns a structured `tool_call` with function name and arguments
10. Your code executes the function and returns the result
11. Result is fed back to the model, which continues its response

```
# Define a tool for the model to use:
tools = [
    {
        'type': 'function',
        'function': {
            'name': 'get_weather',
            'description': 'Get current weather for a city',
            'parameters': {
                'type': 'object',
                'properties': {
                    'city': {'type': 'string', 'description': 'City name'}
                },
                'required': ['city']
            }
        }
    }
]

# Model will call get_weather('London') if user asks about London weather
```

Common Tools Used by Agents

Tool	Purpose
Web Search	Look up current information, news, documentation
Code Interpreter	Write and execute Python code, process data, create charts
File Read/Write	Read input files, write output reports, save results
REST API Calls	Interact with external services (weather, databases, web apps)
Database Query	Run SQL queries against a database
Calculator	Perform precise numeric calculations
Email / Calendar	Send emails, schedule meetings, create events

3.5 Agent Frameworks

LangChain

LangChain is the most popular open-source framework for building LLM-powered applications and agents. It provides modular building blocks:

- Chains — Link multiple LLM calls and tools together in a pipeline
- Agents — LLMs that autonomously decide which tools to use and in what order
- Memory — Built-in memory classes for short-term and long-term storage
- Tools — Integrations with hundreds of services (Google Search, Wikipedia, Python REPL)
- RAG — Easy retrieval-augmented generation with vector stores

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import DuckDuckGoSearchTool
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o')
tools = [DuckDuckGoSearchTool()]

agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = executor.invoke({'input': 'What are the latest trends in API testing?'})
```

CrewAI

CrewAI is a framework for orchestrating multiple AI agents working together as a team (a 'crew'). Each agent has a role, goal, and backstory.

- Role-based agents — Each agent has a defined role (e.g., Researcher, Writer, QA Engineer)
- Task assignment — Specific tasks are assigned to specific agents
- Collaboration — Agents share results and build on each other's work
- Sequential or parallel execution — Tasks can run in order or simultaneously

```

from crewai import Agent, Task, Crew

researcher = Agent(
    role='QA Researcher',
    goal='Research API testing best practices',
    backstory='You are an expert in API testing.'
)

writer = Agent(
    role='Technical Writer',
    goal='Write clear test documentation',
    backstory='You write clear, concise technical documentation.'
)

task1 = Task(description='Research top 5 API testing tools', agent=researcher)
task2 = Task(description='Write a comparison report', agent=writer)

crew = Crew(agents=[researcher, writer], tasks=[task1, task2])
result = crew.kickoff()

```

Framework Comparison

Feature	LangChain	CrewAI
Focus	Single agent + chains	Multi-agent collaboration
Complexity	Medium	Low (high-level abstraction)
Best For	RAG apps, pipelines, single agents	Teams of specialized agents
Tool Support	Hundreds of built-in tools	Uses LangChain tools underneath
Community	Very large, mature	Growing rapidly

04 Multi-Agent Systems

4.1 What are Multi-Agent Systems?

Multi-Agent Systems (MAS) are AI architectures where multiple AI agents work together, each with specialized roles, to accomplish complex tasks that a single agent couldn't handle efficiently alone.

Concept	Explanation
Orchestrator	A master agent that manages and coordinates other agents
Subagents	Specialized agents that each perform a specific subtask
Communication	Agents pass results, instructions, and context to each other
Parallelism	Multiple agents can work on different tasks simultaneously
Specialization	Each agent is optimized for its specific role

Real-World Analogy

Think of a software company: a Project Manager coordinates a Developer, a QA Engineer, a Designer, and a DevOps engineer — each specialized, working in parallel, sharing progress. A multi-agent AI system works the same way.

4.2 Agent Memory

Memory Type	Description
Short-Term (In-Context)	Within the current context window only
Long-Term (External)	Persists beyond a single session
Episodic	Records of past events and interactions
Semantic	Facts and knowledge about the world

How Long-Term Memory Works

12. Conversation or event happens
13. Important information is identified and summarized
14. Summary is converted to an embedding (vector) and stored in a vector database
15. On the next session, a similarity search retrieves relevant memories
16. Retrieved memories are injected into the context window of the new conversation

```
# Simple memory with LangChain:
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain_openai import ChatOpenAI

memory = ConversationBufferMemory()
chain = ConversationChain(llm=ChatOpenAI(), memory=memory)

chain.predict(input='My name is Priya')
chain.predict(input='What is my name?')    # Correctly says 'Priya'
```

4.3 Building Your First Agent

A minimal agent using LangChain with a web search tool:

```
pip install langchain langchain-openai duckduckgo-search
```

```
from langchain.agents import initialize_agent, AgentType
from langchain.tools import DuckDuckGoSearchRun
from langchain_openai import ChatOpenAI

# 1. Define the LLM brain
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# 2. Give the agent tools
tools = [DuckDuckGoSearchRun()]

# 3. Create the agent
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True    # Shows reasoning steps
)

# 4. Run the agent
result = agent.run('What are the top 3 API testing tools in 2025?')
print(result)
```

 When `verbose=True`, you can see the agent's full reasoning: Thought → Action → Observation → Thought → Final Answer. This is the ReAct pattern.

4.4 Real-World Use Cases of Agentic AI

Domain	Use Case	Agents Involved
Software Testing	Autonomous test suite generation and execution	Requirements Analyst, Test Writer, API Tester, Bug Reporter

Research	Deep web research and report writing	Search Agent, Reader Agent, Summarizer Agent, Writer Agent
Customer Support	Handle complex multi-step customer issues	Intake Agent, Database Agent, Resolution Agent, Email Agent
DevOps	Monitor system, detect anomalies, auto-remediate	Monitor Agent, Analyzer Agent, Remediation Agent
Finance	Process invoices, validate data, update records	Extractor Agent, Validator Agent, Database Agent
HR	Screen resumes, schedule interviews, send offers	Screening Agent, Scheduler Agent, Communication Agent

4.5 Limitations & Safety Guardrails

Key Limitations of Agents

Limitation	Description
Hallucination Risk	Agents can make up tool calls or misinterpret results — compounding errors over long runs
Cost	Multi-step agent runs make many LLM calls — can become expensive quickly
Latency	Sequential tool calls take time — agents are slower than single-call chatbots
Reliability	Long agent runs can fail partway through — need robust error handling and retries
Context Overflow	Complex tasks can exhaust the context window
Unpredictability	Agents can take unexpected paths — hard to test and debug

Safety Guardrails

- Human-in-the-Loop (HITL) — Require human approval before irreversible actions (deleting files, sending emails)
- Tool Permission Scoping — Only give agents access to the tools they strictly need
- Output Validation — Validate agent outputs before acting on them
- Rate Limiting — Limit how many tool calls an agent can make per session
- Audit Logging — Log every action and decision an agent makes for review
- Sandboxing — Run agents in isolated environments where they cannot affect production systems
- Kill Switch — Always have a way to stop a runaway agent

 *The golden rule: never give an agent permission to do something irreversible without human confirmation. Build 'checkpoints' into long-running agent workflows.*

05 Team Work, Communication & Professional Skills

5.1 Teamwork & Accountability

What Makes a Good Team Member?

- ✓ Takes ownership of your tasks — do what you say you will do, by when you said
- ✓ Communicates proactively — update your team before they have to ask
- ✓ Respects others' time — be on time, be prepared, stay on topic
- ✓ Helps colleagues — share knowledge, unblock others, celebrate wins together
- ✓ Accepts feedback gracefully — feedback is data, not personal attack
- ✓ Admits mistakes early — hiding a problem makes it worse

Accountability Framework

Behaviour	Low Accountability
When things go wrong	Blame others / external factors
Deadlines	Misses without warning
Updates	Only when asked
Commitments	Says yes, then doesn't deliver

5.2 Listening, Questioning & Probing

Active Listening

- Give full attention — no phone, no multitasking during conversations
- Let the speaker finish before you respond — do not interrupt
- Paraphrase what you heard: 'So if I understand correctly, you mean...'
- Note down key points to remember and action

Types of Questions

Question Type	Purpose
Open Question	Gets detailed, conversational answers
Closed Question	Gets a specific yes/no or short answer
Probing Question	Digs deeper into an answer just given
Clarifying Question	Removes ambiguity

Hypothetical Question	Explores possibilities
-----------------------	------------------------

Probing Technique — The 5 Whys

When you encounter a problem, keep asking 'why' until you reach the root cause:

```

Problem: The test failed.
Why?    → The API returned 500.
Why?    → The database query timed out.
Why?    → The query was missing an index.
Why?    → The new table was created without reviewing the query plan.
Why?    → There is no mandatory code review step for schema changes.
Root Cause: Missing process for schema change review.

```

5.3 Meeting Etiquette

Before the Meeting	During the Meeting
Know the agenda before joining	Be on time — log in 1-2 min early
Prepare your updates/questions	Camera on for important meetings
Test your audio/video	Mute when not speaking
Have relevant docs open	Ask questions in the right slot
Decline politely if not needed	Avoid side-conversations

Speaking Up in Meetings

- Prepare one or two points you want to contribute before the meeting starts
- Use the chat box if you are not comfortable speaking up immediately
- Keep contributions concise — state the point, give one supporting detail, stop
- If you disagree, do it respectfully: 'I see it slightly differently — can I share my view?'

5.4 Interview Skills

The STAR Method for Behavioural Questions

STAR is a structured way to answer behavioural interview questions (questions that start with 'Tell me about a time when...')

Letter	Stands For
S	Situation

T	Task
A	Action
R	Result

Question: 'Tell me about a time you found a critical bug close to release.'

S: During a release sprint, we had 2 days before go-live on a payment feature.

T: I was responsible for final regression testing of the checkout API.

A: I wrote an exploratory test with an expired credit card token and discovered the API returned 200 OK instead of 402 — charging nothing but recording success.

R: I raised a P1 defect immediately. The dev team fixed it in 4 hours.

The release was delayed by 1 day but we prevented a major revenue loss issue.

Common QA Interview Topics

- SDLC and STLC — explain each phase and your role
- Types of testing — functional, regression, smoke, sanity, exploratory, API, performance
- Bug life cycle — New → Assigned → In Progress → Fixed → Retesting → Closed
- What makes a good test case — clear preconditions, steps, expected result
- Agile / Scrum — sprints, ceremonies, your role as QA in a sprint
- API testing — HTTP methods, status codes, tools (Postman), assertions
- Automation basics — frameworks, what to automate vs not

Email & Grammar in Professional Communication

Do	Don't
Use a clear, specific subject line	Use vague subjects like 'Question' or 'Issue'
State your purpose in the first sentence	Bury the ask at the bottom of a long email
Use short paragraphs and bullet points	Send walls of unbroken text
Proofread before sending — check spelling	Send with typos and grammatical errors
Use professional greeting: Hi [Name]	Use 'Hey' or start with no greeting
End with a clear next step or ask	Leave the reader unsure what to do next
Reply within 24 business hours	Leave emails unanswered for days

06 Quick Reference Cheat Sheet

Gen AI Key Terms

Term	Definition
LLM	Large Language Model — a neural network trained on text to generate text
Token	Smallest unit of text processed by an LLM (~0.75 words)
Context Window	Max tokens an LLM can process at once (input + output)
Transformer	Neural network architecture behind all modern LLMs
Self-Attention	Mechanism that relates words to each other in context
Temperature	Controls output randomness (0=focused, 1=creative)
Hallucination	Model confidently generates false information
RAG	Retrieval Augmented Generation — retrieves docs before generating
Fine-tuning	Additional training on domain-specific data
RLHF	Reinforcement Learning from Human Feedback — aligns model behavior
Embedding	Numerical vector representation of text for similarity search
Prompt Engineering	Designing prompts to get optimal LLM outputs

Agentic AI Key Terms

Term	Definition
AI Agent	Autonomous AI system that plans and takes actions to achieve a goal
Orchestrator	Master agent that coordinates other specialist subagents
Tool Use / Function Calling	Agent's ability to invoke external tools (APIs, code, search)
ReAct Pattern	Reasoning + Acting — agent thinks, acts, observes, repeats
LangChain	Python framework for building LLM apps and agents
CrewAI	Framework for multi-agent collaboration with role-based agents
Short-Term Memory	In-context memory within a single conversation session
Long-Term Memory	External memory stored in a database, persists across sessions

Vector Database	Database that stores embeddings for semantic similarity search
HITL	Human-in-the-Loop — human approval required for certain agent actions

Prompting Quick Guide

```
ZERO-SHOT: Just ask directly with no examples.  
FEW-SHOT:  Provide 2-5 examples, then ask.  
COT:       Add 'Think step by step' to complex reasoning prompts.  
ROLE:      Start with 'You are a [role]. ...' for expert-style output.  
FORMAT:    End with 'Format as JSON / bullet points / table / markdown'.  
CONSTRAIN: Add 'in under 200 words' or 'in simple English for beginners'.
```